

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT Student name to your left: _____

PRINT Student name to your right: _____

You have 110 minutes. There are 8 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	8	Total
Points:	12	6	10	12	10	16	20	14	100

For questions with **circular bubbles**, select only one choice (there is only one correct answer).

- ☐ Unselected option (completely unfilled)
- ☒ Don't do this (it will be graded as incorrect)
- ☒ Only one selected option (completely filled)

For questions with **square boxes**, you may select one or more choices (select all that apply).

- ☐ You can select
- ☐ multiple squares
- ☒ Don't do this (it will be graded as incorrect)

- Anything you write outside the answer boxes or you ~~cross-out~~ will not be graded. If you write multiple answers or your answer is ambiguous, we will grade the **worst** interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.
- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B/61BL style guidelines. For example, any method, constructor, or if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Read the honor code below and sign your name.

By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.
--

SIGN your name: _____

Q1 Map

(12 points)

Fill in `rankedAbove`, which takes in a list of `Player`s and returns a map from each `Player` to their rank. The player with the highest score has rank 1, the player with the next-highest score has rank 2, and so on.

For example, if we have a list of players with `scores` of 500, 800, 1200, and 100, then these players would have ranks 3, 2, 1, and 4, respectively, and `rankedAbove` would return the following `Map`:

```
{ Player with score 500 : 3,
  Player with score 800 : 2,
  Player with score 1200 : 1,
  Player with score 100 : 4 }
```

Assume no two players have the same score.

You may not need all lines. Do not worry about efficiency.

```
public class Player {
    public double score;

    public static Map<Player, Integer> rankedAbove(List<Player> players) {
        _____ results = _____;

        for (Player _____) {
            _____
            _____
            _____
            _____
            _____
            _____
            _____

        }

        return results;
    }
}
```

Q2 Equals

(6 points)

Fill in the `equals` method below. Two `GoatMeasurement` objects are considered equal if and only if they have the same `name`, and their `sizes` differ by at most 0.1. Assume `name` is never null.

Hint: You may find `Math.abs` useful.

Syntax hint: `if (o instanceof Dog uddaDog)`

```
public class GoatMeasurement {
    public String name;
    public double size;

    @Override
    public boolean equals(Object o) {

        if (_____ ) {

            return (_____
                _____);

        }

        return _____;

    }
}
```

Q3 *LinkedListDeque61B* Addition

(10 points)

Consider the `LinkedListDeque61B` from Project 1. Implement `add`, which adds `x` to position `idx` in the deque.

For example, `add(1, "zoo")` changes `[ant, bee, cat]` to `[ant, zoo, bee, cat]`.

Assume that `idx` is always within `[0, size)`.

You may not use any if statements. You may not need all lines.

```
public class LinkedListDeque61B<T> implements Deque61B<T> {
    private Node sentinel; // Circular linked list topology
    private int size;

    private class Node {
        public T value;
        public Node prev, next;
        public Node(T value, Node prev, Node next) {
            this.value = value;
            this.prev = prev;
            this.next = next;
        }
    }

    ...

    public void add(int idx, T x) {

        _____

        _____

        _____

        _____

        _____

        _____

        _____

        _____

    }
}
```

Q4 A Default Method

(12 points)

Implement `rotateToFront`, which rotates a deque to the left, such that the first element equal to `x` becomes the first element in the deque. If `x` is not in the deque, the order of the deque should be unchanged. The method should return `true` if `x` is found, and `false` otherwise.

For example, `rotateToFront(9)` changes `[1, 2, 9, 3, 4, 9]` to `[9, 3, 4, 9, 1, 2]`.

For example, `rotateToFront(4)` changes `[1, 2, 9, 3, 4, 9]` to `[4, 9, 1, 2, 9, 3]`.

Assume that `x` is not `null` and that the deque contains no `nulls`. You may use any methods from the `Deque61B` interface (see reference card). **You may not use `new`.** You may not need all lines.

```
public interface Deque61B<T> {
```

```
    ...
```

```
    default boolean rotateToFront(T x) {
```

```
    }
```

```
}
```

Q5 Resize

(10 points)

Consider the `ArrayDeque61B` from Project 2. Implement `resize`, which resizes the backing array to a length of `capacity` and places the items in order, **starting at index 0**.

For example, after calling `resize(10)`, `items` should be of length 10, and `nextFirst` and `nextLast` should be updated appropriately.

You may use any methods from the `Deque61B` interface (see reference card). Assume that `capacity` is always greater than or equal to the number of items in the `Deque61B`.

Syntax hint: `Glorp[] arrayOfGenerics = (Glorp[]) new Object[61];`

You may not need all lines.

```
public class ArrayDeque61B<T> implements Deque61B<T> {
    private T[] items;
    private int size;          // Total number of items in the deque
    private int nextFirst;     // Index where addFirst will add an item if
                                // called next
    private int nextLast;     // Index where addLast will add an item if
                                // called next
    ...

    /**
     * Resizes array so that its length is equal to capacity.
     * The first item in the ArrayDeque61B should be at index 0.
     */
    private void resize(int capacity) {

        _____

        _____

        _____

        _____

        _____

        _____

        _____

        _____

    }
}
```

Q6 Prime Factorization

(16 points)

The `primeFactors` method takes an `int x > 1` and returns a list of `x`'s prime factors in increasing order. For example, `primeFactors(60)` should return `[2, 2, 3, 5]`.

Q6.1 (4 points) Fill in `smallestPrimeFactor`, a helper method that returns the smallest prime factor of `x`. For example, `smallestPrimeFactor(60)` is 2, and `smallestPrimeFactor(63)` is 3.

```
public static int smallestPrimeFactor(int x) {  
    _____;  
    while (true) {  
        if (_____) {  
            _____;  
        }  
        _____;  
    }  
}
```

Q6.2 (8 points) Fill in `addAllPrimeFactors`, a helper method that adds all prime factors of `x` to `factors`, **using recursion**. You may not need all lines.

You may use `smallestPrimeFactor` and assume it works as described.

```
public static void addAllPrimeFactors(int x, List<Integer> factors) {  
    if (_____) {  
        _____;  
    }  
    _____;  
    _____;  
    _____;  
}
```

Q6.3 (4 points) Fill in `primeFactors`, which returns all prime factors of `x` in increasing order. You may use the above helper method(s) and assume they work as described.

```
public static List<Integer> primeFactors(int x) {  
    _____;  
    _____;  
    return _____;  
}
```

Q7 Ejection

(20 points)

The **eject** methods remove all appearances of **int y** from the provided **IntList**.

Q7.1 (10 points) Fill in **ejectND**, which is **non-destructive**.

```
public static IntList ejectND(IntList x, int y) {  
  
    if (_____) {  
  
        _____;  
    }  
  
    if (_____) {  
  
        return _____;  
    }  
  
    return new _____;  
}
```

Q7.2 (10 points) Fill in **ejectD**, which is **destructive**. You may not use **new**.

Note: This is one of the most difficult questions on the exam.

```
public static IntList ejectD(IntList x, int y) {  
  
    if (_____) {  
  
        _____;  
    }  
  
    _____;  
  
    if (x.first == y) {  
  
        return _____;  
    }  
  
    return _____;  
}
```


Q8 LEQIterator

(14 points)

Fill in `LEQIterator`, which iterates over all numbers in an `IntList` that are less than or equal (LEQ) to the integer at the front of the list. For example:

```
IntList L = IntList.of(25, 30, 10, 5, 90, 25, 80);
Iterator<Integer> i = new LEQIterator(L);
while (i.hasNext()) {
    System.out.println(i.next()); // prints 25, 10, 5, 25
}
```

Assume that `L` is not null. You do not need to throw any exceptions. You may not need all lines.

```
public class LEQIterator implements Iterator<Integer> {

    private _____;

    private _____;

    public LEQIterator(IntList L) {
        _____;
        _____;
    }

    public boolean hasNext() {
        return _____;
    }

    public Integer next() {
        _____;
        _____;

        while (_____ &&
               _____) {
            _____;
        }

        _____;
    }
}
```

You have reached the end of the exam! Nothing on this page is worth any credit.

PNH Question

My 3rd grade daughter suggested that I should provide an opportunity for a reward for my CS61B students. She told me, "If they're good, you should let them watch..."

What do you think she suggested?

Feedback

Leave any feedback, comments, and/or drawings in the box below!